Al-Furqan — Daily Development Report

March 21, 2026 (Saturday — عيد الفطر)

Project: Al-Furqan (الفرقان) — Axiom-Anchored Neuro-Symbolic Reasoning Engine
Repository: https://gitlab.variiance.com/ai/al-furqan
Day Summary: Massive execution day — 7 sprints completed, 2 MCP skills built, 647 total tests, full security hardening
Starting state: Sprint 2 complete (205 tests, monolithic engine)
Ending state: Sprints 3-6 complete + RaaS + Memory skills (647 tests, layered architecture)

Table of Contents

---

1. Executive Summary

What Was Accomplished

In a single day, the Al-Furqan project went from a monolithic Sprint 2 codebase (205 tests) to a fully layered, security-hardened architecture with:

- **Engine refactored** into separate modules: axioms, models, pipeline, prompts, gates, chains, symbolic
- **4 independent gate implementations** with deterministic scoring
- **Z3 symbolic verification** with per-gate checking (3 axioms + 2 proofs encoded)
- **Knowledge Base layer** with Quran, Hadith, Fiqh collections + graph + retriever
- **Security hardening** with 5 independent security modules
- **Orchestrator** connecting all layers with security wired in
- **Furqan RaaS** (Reasoning-as-a-Skill) — MCP server with 5 tools
- **Furqan Memory** — client-side memory skill with SQLite + ChromaDB
- **647 total tests** (560 engine + 87 skills)
- **8 architecture/design documents** created

Agents Spawned

Multiple subagents were spawned throughout the day to parallelize work:

Agent Type	Tasks	Key Output
Engine Specialist	Sprint 3A refactor, 4A gates, 4B chains	20+ new modules

KB Specialist	Sprint 3B embeddings, 3C collections	9 KB modules
Symbolic AI	Sprint 4C Z3 verification	3 symbolic modules
Security	Sprint 6 security hardening	5 security modules
Orchestrator	Sprint 5A integration	Orchestrator + EvaluationResult
RaaS Builder	Furqan RaaS skill	Full MCP server + 31 tests
Memory Builder	Furqan Memory skill	Full MCP server + 56 tests
Documentation	Architecture docs, plans, PDFs	8 documents
Test Engineer	Test suites for all modules	442 new tests

---

2. Chronological Timeline

All times in UTC.

Phase 1: Design Documents (04:00–08:00)

Time (UTC)	Activity
~04:22	Created `FURQAN-REASONING-AS-A-SKILL-v1.0.md` — RaaS architecture document
~04:31	Generated RaaS PDF with rendered diagrams
~04:43	Created `FURQAN-MEMORY-SKILL-v1.0.md` — Memory skill architecture
~04:43	Generated Memory skill PDF
~05:03	Created `FURQAN-AXIOM-SECURITY-POLICY-v1.0.md` — Security policy document
~05:03	Generated Security policy PDF
~07:15	Created `AL-FURQAN-KNOWLEDGE-GRAPH-DOCS.md` — Knowledge graph documentation
~07:15	Generated Knowledge Graph PDF
~07:59	Created `AL-FURQAN-IMPLEMENTATION-PLAN-v1.0.md` — Full implementation plan
~07:59	Generated Implementation Plan PDF

Phase 2: Engine Core (Sprint 3A) (08:00–09:15)

Time (UTC)	Activity
~08:08	Created `engine/axioms.py` — extracted axioms with SHA-256 hashing
~08:09	Created `engine/prompts.py` — all prompt templates + sanitization
~08:10	Created `engine/pipeline.py` — evaluation pipeline with JSON repair
~09:12	Created `engine/models.py` — all data models with model metadata

Phase 3: Symbolic + Gates + Chains (Sprint 4) (09:15–12:05)

Time (UTC)	Activity
~09:47	Created `engine/symbolic/formal_axioms.py` — Z3-encoded axioms
~09:47	Created `engine/symbolic/predicate_extractor.py` — chain → Z3 mapping
~09:47	Created `engine/gates/base.py` + all 4 gate implementations
~09:47	Created `engine/chains/` — definitions, executor, scorer
~09:48	Created `engine/symbolic/__init__.py`
~11:12	Updated gate implementations (origin_aware, structural_consistency)
~11:36	Updated `source_integrity.py` — divine=100 scoring decision
~12:03	Updated `symbolic/verifier.py` — added per-gate verification

Phase 4: Skills Implementation Plan (12:00–13:40)

Time (UTC)	Activity
~13:38	Created `FURQAN-SKILLS-IMPLEMENTATION-PLAN-v1.0.md`
~13:39	Generated Skills Implementation Plan PDF

Phase 5: Security Hardening (Sprint 6) (14:06–14:08)

Time (UTC)	Activity
~14:06	Created `engine/security/__init__.py` — security module exports
~14:06	Created `engine/security/integrity.py` — axiom hash verification
~14:06	Created `engine/security/prompt_guard.py` — injection detection
~14:07	Created `engine/security/output_validator.py` — output validation
~14:07	Created `engine/security/adapter_sandbox.py` — Z3 contradiction checking
~14:07	Created `engine/security/audit.py` — privacy-preserving audit logs
~14:08	Created `api/orchestrator.py` — full pipeline orchestrator

Phase 6: MCP Skills (15:47–15:56)

Time (UTC)	Activity
~15:47	Created `furqan-raas/` — complete RaaS MCP server
~15:49	Created `furqan-memory/` — memory skill structure
~15:50	Created `furqan-memory/storage/sqlite_store.py` — 4-table schema

~15:51	Created `furqan-memory/memory_manager.py` — core operations
~15:52	Created `furqan-memory/mcp_server.py` — MCP server with 5 tools
~15:56	Created `furqan-memory/storage/vector_store.py` — ChromaDB integration

Phase 7: Documentation (16:08–16:17)

Time (UTC)	Activity
~16:12	Created `docs/SPRINT-3-5-ENGINE-DOCS.md`
~16:13	Created `docs/SPRINT-6-SECURITY-DOCS.md`
~16:15	Created `docs/FURQAN-RAAS-DOCS.md`
~16:16	Created `docs/FURQAN-MEMORY-DOCS.md`
~16:17	Created `docs/DAILY-REPORT-2026-03-21.md` (this file)

---

3. Sprint Execution Details

Sprint 3A: Engine Refactor + Model Metadata

Duration: ~08:00–09:15 UTC

Extracted the monolithic `core/reasoning_engine.py` into a clean modular structure:

Module	Purpose	Key Content
`engine/axioms.py`	Immutable axioms	AXIOM_VERSION, SHA-256 hash, 4 constant strings
`engine/models.py`	Data structures	6 classes (SystemType, GateResult, GateScore, Verdict, DualPerspectiveVerdict, InformationalResponse)
`engine/pipeline.py`	Evaluation flow	EvaluationPipeline with Scan→Mirror→Verdict→Correct
`engine/prompts.py`	Prompt templates	6 prompt builders + input sanitization

Sprint 3B-3E: KB Infrastructure

All KB modules created:

- `kb/embeddings.py` — CamelBERT + MiniLM with fallback
- `kb/collections/quran.py`, `hadith.py`, `fiqh.py` — 3 collections
- `kb/retriever.py` — UnifiedRetriever with deduplication
- `kb/graph/schema.py`, `store.py`, `traversal.py` — knowledge graph
- `kb/knowledge_linker.py` — graph-enhanced retrieval
- `kb/ingestion/` — 3 ingestion pipelines

Sprint 4A: Gate Decomposition

4 independent gate implementations, each with:

- Abstract base class (`gates/base.py`)
- Deterministic scoring (pure Python, no LLM)
- Chain questions for LLM fact extraction
- Survive/Fail threshold

Key scoring decisions:

- Source Integrity: divine=**100** (changed from 90 to 100)
- Origin Aware: BINARY gate (100 or 0, no range)

Sprint 4B: Chains + Scorer

- `chains/definitions.py` — 16 total chain questions across 4 gates
- `chains/executor.py` — LLM-driven fact extraction with context accumulation
- `chains/scorer.py` — DeterministicScorer (same input → same score, guaranteed)

Sprint 4C: Z3 Symbolic Verification

- `symbolic/formal_axioms.py` — 3 axioms + 2 proofs encoded in Z3
- `symbolic/predicate_extractor.py` — Maps evaluation data → Z3 predicates
- `symbolic/verifier.py` — Full verifier with per-gate independent checking

Key innovation: Per-gate Z3 verification — Instead of one holistic check, each gate gets its own Z3 verification with only relevant predicates. This provides gate-specific formal proofs.

Sprint 5A: Orchestrator

- `api/orchestrator.py` — Central pipeline with security wired in
- `EvaluationResult` — complete result with verdict + sources + Z3 + audit

Sprint 6: Security Hardening

5 security modules created and integrated:

1. `IntegrityVerifier` — SHA-256 axiom hash checking (verify_or_die)
2. `PromptGuard` — 12 injection patterns detected
3. `OutputValidator` — verdict structure + range validation
4. `AdapterSandbox` — Z3-backed domain axiom contradiction checking
5. `AuditLogger` — privacy-preserving logs (question hash, not text)

---

4. Architecture Documents Created

All documents created on March 21, 2026:

Document	Time	Size	Description
`FURQAN-REASONING-AS-A-SKILL-v1.0.md`	04:22	25.5 KB	RaaS architecture + API reference
`FURQAN-MEMORY-SKILL-v1.0.md`	04:43	32.0 KB	Memory skill design + schema
`FURQAN-AXIOM-SECURITY-POLICY-v1.0.md`	05:03	46.1 KB	Security policy + threat model

`AL-FURQAN-KNOWLEDGE-GRAPH-DOCS.md`	07:15	11.9 KB	Knowledge graph schema docs
`AL-FURQAN-IMPLEMENTATION-PLAN-v1.0.md`	07:59	34.8 KB	Task-level implementation plan
`FURQAN-SKILLS-IMPLEMENTATION-PLAN-v1.0.md`	13:38	25.1 KB	Skills development roadmap
`SPRINT-3-5-ENGINE-DOCS.md`	16:12	31.4 KB	Engine technical reference
`SPRINT-6-SECURITY-DOCS.md`	16:13	17.9 KB	Security hardening reference
`FURQAN-RAAS-DOCS.md`	16:15	15.1 KB	RaaS product documentation
`FURQAN-MEMORY-DOCS.md`	16:16	17.5 KB	Memory skill documentation
`DAILY-REPORT-2026-03-21.md`	16:17	—	This report

PDF versions generated for: RaaS, Memory, Security Policy, Knowledge Graph, Implementation Plan, Skills Plan.

---

5. Test Count Evolution

Stage	Count	Delta	Notes
Sprint 2 (start of day)	205	—	Auth, API, core engine
+ Sprint 3A (engine refactor)	~220	+15	axiom, model, pipeline tests
+ Sprint 3B-3E (KB)	~357	+137	embeddings, collections, graph, retriever
+ Sprint 4A-4D (gates, chains, Z3)	~437	+80	gate tests, chain tests, Z3 tests
+ Sprint 5A (orchestrator)	~459	+22	orchestrator tests
+ Sprint 6 (security)	~529	+70	security module tests
+ Performance/integration	~560	+31	end-to-end, performance
Engine subtotal	**560**		
+ RaaS tests	591	+31	MCP server tests
+ Memory tests	647	+56	MCP + manager + store + vector
Grand Total	**647**	**+442**	

---

6. Key Technical Decisions

Decision 1: divine = 100 (not 90)

What: Source Integrity gate scoring for divine sources changed from 90 to 100.

Why: A divine source (Quran) is the highest possible source type. Giving it 90 implies there's something higher (110? impossible). The scale is 0-100, divine is the maximum.

Impact: `SOURCE_TYPE_SCORES["divine"] = 100` in `engine/gates/source_integrity.py`.

Decision 2: Per-Gate Z3 Verification

What: Instead of one holistic Z3 check, each gate gets its own independent verification.

Why:

- A holistic check can't pinpoint which gate caused the contradiction
- Per-gate verification provides gate-specific formal proofs
- Better diagnostic value for debugging and explanation

Implementation: `SymbolicVerifier.verify_per_gate()` runs 4 separate Z3 checks with gate-specific predicates.

Decision 3: Provenance Enforcement

What: The engine must track model_provider, model_name, model_temperature, and raw responses for every evaluation.

Why: Reproducibility and audit trail. If a verdict seems wrong, we need to know exactly which model produced it and with what parameters.

Implementation: `model_provider`, `model_name`, `model_temperature`, `raw_scan_response`, `raw_mirror_response`, `raw_verdict_response` fields on `Verdict`.

Decision 4: Privacy-First Audit Logging

What: Audit logs store SHA-256 hash of questions, never the plaintext.

Why: Audit trail needed for security, but questions may contain sensitive personal information. Hash is sufficient for correlation without privacy risk.

Implementation: `AuditLogger.hash_question()` → `question_hash` field in log entries.

Decision 5: Client-Side Only Memory

What: Furqan Memory runs entirely on the user's device with no cloud sync.

Why: User questions and verdicts are sensitive. Cloud storage creates privacy risk. Memory is more useful when it's fast (local) than when it's shared (cloud).

---

7. Bug Fixes & Adjustments

Fix 1: Key Mapping in Gate Evaluation

Problem: Chain results used inconsistent keys (`is_verifiable` vs `verifiable` , `has_logical_gaps` vs `logical_gaps` , `acknowledges_transcendent` vs `acknowledges_transcendence`).

Fix: All gate implementations now check both old and new key names with fallback:

```
verifiable = bool(chain_results.get("verifiable", chain_results.get("is_verifiable",
False)))
```

Fix 2: Test Adjustments for Scoring Changes

Problem: When divine score changed from 90 to 100, several tests expected old values.

Fix: Updated test assertions to match new scoring logic.

Fix 3: Vector Store Metadata Safety

Problem: ChromaDB only accepts `str` , `int` , `float` , `bool` as metadata values. Complex types caused errors.

Fix: `MemoryVectorSearch._safe_metadata()` converts complex types to JSON strings and drops `None` values.

---

8. Model Selection & Benchmarks

Selected Model: `qwen3.5-397b-a17b` (Qwen 3.5 MoE)

Why selected:

- Best Arabic performance among available models
- MoE architecture (397B total, 17B active) — efficient inference
- Strong at structured output (JSON extraction)
- Good at following complex multi-step instructions
- Available via Alibaba DashScope API

Architecture Support

The engine is model-agnostic through `EvaluationPipeline(llm_call)` :

```
# Any callable that takes prompt string → returns string
pipeline = EvaluationPipeline(llm_call=my_llm_function)
```

Supported providers via `providers/llm_layer.py` :

- Anthropic (Claude)
- Alibaba DashScope (Qwen)
- Ollama (local models)

---

9. Business Context

Market Analysis

Al-Furqan is positioned as the **first axiom-anchored reasoning engine** — not a chatbot, not a fatwa generator, but a formal verification system for evaluating any claim or system against immutable axioms.

Competitive Differentiation

Feature	Al-Furqan	Islamic Chatbots	General Reasoning
Formal Z3 proofs	✅	❌	❌
Deterministic scoring	✅	❌	❌
Source grounding	✅	Partial	❌
Model agnostic	✅	❌	Partial
Axiom integrity checking	✅	❌	❌
MCP skill ecosystem	✅	❌	❌

QLP v3.0 Connection

Al-Furqan is a key component of the QLP v3.0 (قلب) vision — providing the reasoning layer for digital sovereignty. The local-first architecture (edge deployment with local KB) aligns with QLP's privacy-first principles.

---

10. Files Modified/Created

New Directories Created

```
src/al_furqan/engine/           (Sprint 3A)
src/al_furqan/engine/gates/     (Sprint 4A)
src/al_furqan/engine/chains/    (Sprint 4B)
src/al_furqan/engine/symbolic/  (Sprint 4C)
src/al_furqan/engine/security/  (Sprint 6)
src/al_furqan/kb/               (Sprint 3B-3E)
src/al_furqan/kb/collections/   (Sprint 3C)
src/al_furqan/kb/graph/         (Sprint 3D)
src/al_furqan/kb/ingestion/     (Sprint 3C)
furqan-raas/                    (RaaS Skill)
furqan-memory/                  (Memory Skill)
```

New Source Files (by sprint)

Sprint 3A — Engine Core (4 files):

- `engine/axioms.py`, `engine/models.py`, `engine/pipeline.py`, `engine/prompts.py`

Sprint 3B — Embeddings (1 file):

- `kb/embeddings.py`

Sprint 3C — Collections (3 files):

- `kb/collections/quran.py`, `kb/collections/hadith.py`, `kb/collections/fiqh.py`

Sprint 3C — Ingestion (3 files):

- `kb/ingestion/ingest_quran.py`, `kb/ingestion/ingest_hadith.py`, `kb/ingestion/ingest_fiqh.py`

Sprint 3C — Retrieval (2 files):

- `kb/retriever.py`, `kb/knowledge_linker.py`

Sprint 3D — Graph (3 files):

- `kb/graph/schema.py`, `kb/graph/store.py`, `kb/graph/traversal.py`

Sprint 4A — Gates (5 files):

- `engine/gates/base.py`, `engine/gates/source_integrity.py`, `engine/gates/structural_consistency.py`, `engine/gates/mediation_zeroing.py`, `engine/gates/origin_aware.py`

Sprint 4B — Chains (3 files):

- `engine/chains/definitions.py`, `engine/chains/executor.py`, `engine/chains/scorer.py`

Sprint 4C — Symbolic (3 files):

- `engine/symbolic/formal_axioms.py`, `engine/symbolic/predicate_extractor.py`, `engine/symbolic/verifier.py`

Sprint 5A — Orchestrator (1 file):

- `api/orchestrator.py`

Sprint 6 — Security (5 files):

- `engine/security/integrity.py`, `engine/security/prompt_guard.py`, `engine/security/output_validator.py`, `engine/security/adapter_sandbox.py`, `engine/security/audit.py`

RaaS Skill (3 files):

- `furqan-raas/src/furqan_raas/__init__.py`, `__main__.py`, `mcp_server.py`

Memory Skill (6 files):

- `furqan-memory/src/furqan_memory/__init__.py`, `__main__.py`, `mcp_server.py`, `memory_manager.py`, `storage/sqlite_store.py`, `storage/vector_store.py`

New Test Files (40 files)

```
tests/test_adapter_sandbox.py          (8 tests)
tests/test_audit_logger.py             (9 tests)
tests/test_chain_executor.py           (6 tests)
tests/test_deterministic_scorer.py     (9 tests)
tests/test_embeddings.py               (17 tests)
tests/test_end_to_end.py               (8 tests)
tests/test_engine_integration.py       (9 tests)
tests/test_fiqh_collection.py          (18 tests)
```

```
tests/test_gate_mediation_zeroing.py   (8 tests)
tests/test_gate_origin_aware.py        (6 tests)
tests/test_gate_source_integrity.py    (12 tests)
tests/test_gate_structural_consistency.py (8 tests)
tests/test_graph_integration.py        (12 tests)
tests/test_graph_store.py              (30 tests)
tests/test_hadith_collection.py        (13 tests)
tests/test_integrity_verifier.py       (13 tests)
tests/test_kb_integration.py           (6 tests)
tests/test_knowledge_linker.py         (12 tests)
tests/test_orchestrator.py             (22 tests)
tests/test_output_validator.py         (14 tests)
tests/test_performance.py              (6 tests)
tests/test_predicate_extractor.py      (12 tests)
tests/test_prompt_guard.py             (19 tests)
tests/test_quran_collection.py         (14 tests)
tests/test_retriever.py                (15 tests)
tests/test_security.py                 (7 tests)
tests/test_symbolic_verifier.py        (14 tests)
tests/test_z3_axioms.py                (19 tests)
furqan-raas/tests/test_mcp_server.py   (31 tests)
furqan-memory/tests/test_mcp_memory.py (16 tests)
furqan-memory/tests/test_memory_manager.py (14 tests)
furqan-memory/tests/test_sqlite_store.py (18 tests)
furqan-memory/tests/test_vector_search.py (8 tests)
```

Documents Created (11 files)

See Section 4 for full list.

Summary Statistics

Metric	Value
Sprints completed	7 (3A, 3B-3E, 4A-4D, 5A, 6)
New source files	~45
New test files	~33
Total tests	647 (up from 205)
New tests	+442
Documents created	11
PDFs generated	6
MCP skills built	2 (RaaS + Memory)
Security modules	5
Z3 axioms encoded	3 axioms + 2 proofs

Gate implementations	4
Chain questions	16
Lines of code (estimated)	~8,000+ new

Al-Furqan Daily Report — March 21, 2026
Variiance R&D — The Criterion Project
"The engine judges. The knowledge informs. Neither depends on the other."